

CNN Architecture and LeNet-5

Contents

1. Overview	2
2. Prerequisites	2
3. The general CNN architecture	3
3.1 What can be varied	4
4. LeNet-5	4
4.1 Background	4
4.2 The layers	4
4.3 The shapes, layer by layer	5
4.4 Why “5”	7
5. Counting the parameters	8
6. LeNet-5 in Keras	9
7. Summary	12
8. Sources	12
9. Key terms	12

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Convolution operation and feature maps ([Note DL-042](#)); Padding and strides ([Note DL-043](#)); Pooling ([Note DL-044](#)).
- **Leads to:** Image classification with a CNN (cats vs dogs) ([Note DL-049](#)); Pretrained models and ImageNet ([Note DL-051](#)).

1. Overview

Key point: A CNN stacks blocks of convolution and pooling, flattens the result, and finishes with fully connected layers and an output layer. LeNet-5 (LeCun et al. 1998) is the classic example: two convolution-and-pooling blocks, then layers of 120, 84 and 10 nodes, about 60,000 parameters in all.

The parts were built earlier: the [convolution operation](#) (sliding a small grid of weights over an image), [padding and strides](#) and [pooling](#) (shrinking a feature map). This Note puts them together into a whole network, first in general and then in the first CNN that solved a real problem.

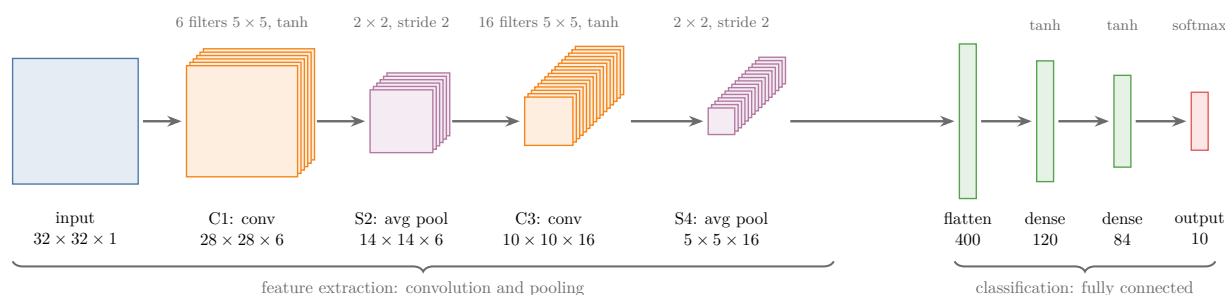


Figure 1: LeNet-5: two blocks of convolution (tanh) and average pooling turn a 32×32 image into 16 maps of 5×5 ; these are flattened to 400 numbers and passed through fully connected layers of 120, 84 and 10 nodes

Figure 1 shows LeNet-5. This Note covers:

- the general CNN architecture and what can be varied in it (section 3);
- the LeNet-5 layers and their shapes (section 4);
- the parameter count (section 5);
- LeNet-5 in Keras, trained on MNIST (section 6).

2. Prerequisites

- [The convolution operation](#), [padding and strides](#) and [max pooling](#).

- **The MNIST network:** **Flatten** (G-788), Dense layers, softmax output and counting parameters (G-1448).
- **Tanh** and **ReLU**, two activation functions (functions applied to each node's sum).

3. The general CNN architecture

Key point: Input, then one or more blocks of convolution + ReLU + pooling, then Flatten, then any number of fully connected layers, then the output layer.

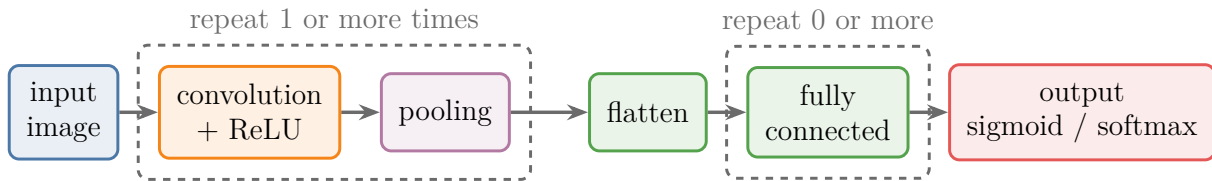


Figure 2: The common pattern of a CNN: convolution and pooling blocks extract features; flatten and fully connected layers classify

Almost every CNN follows the pattern of Figure 2. The sequence of layers is the **CNN architecture** (G-403):

1. **Input:** an image, for example an RGB image of $32 \times 32 \times 3$.
2. **Convolution layer:** (G-480) a set of **filters** (G-777) (kernels), say 3, each with 3 channels (layers of the image, such as red, green and blue) because the input has 3. The output is a volume of feature maps with 3 channels, one per filter.
3. **Non-linearity:** an activation function such as ReLU applied to every number of the feature maps.
4. **Pooling layer** (G-1520): shrinks the volume.
5. **Repeat** steps 2–4 as many times as needed: a second convolution, a second pooling, and so on.
6. **Flatten:** turn the final 3D volume into one long 1D vector of numbers.
7. **Fully connected layers** (G-583): the Dense layers of an ANN, as many as the problem needs.
8. **Output layer** (G-1424): one node with sigmoid ([sigmoid](#), a function that squeezes a number into 0 to 1) for binary classification, or one node per class with softmax (a function that turns the scores into probabilities that add to 1) for multi-class classification.

The convolution and pooling blocks extract the features; the fully connected part uses them to classify. The CS231n notes write the same pattern as $\text{INPUT} \rightarrow [[\text{CONV} \rightarrow \text{RELU}] \times N \rightarrow \text{POOL?}] \times M \rightarrow [\text{FC} \rightarrow \text{RELU}] \times K \rightarrow \text{FC}$.

3.1 What can be varied

Key point: The number of convolution layers and filters, filter size, stride, padding, the fully connected part, the activation, and extras such as dropout and batch normalisation.

Different CNN architectures come from different choices for:

- the number of convolution layers, and the number of filters in each;
- the filter size, the stride and whether there is padding;
- the number of fully connected layers and of nodes in each;
- the activation function;
- whether [dropout](#) or [batch normalisation](#) is used.

Over the years, competitions such as **ImageNet** (G-920), a hard image classification task with many classes, have produced a series of famous architectures: LeNet, AlexNet, GoogLeNet, VGGNet, ResNet and Inception. All of them follow the pattern above with different choices. LeNet came first.

4. LeNet-5

Key point: Yann LeCun’s network for reading handwritten digits: input 32×32 , convolution with 6 filters of 5×5 , average pooling, convolution with 16 filters of 5×5 , average pooling, then 120, 84 and 10 nodes, with tanh activations.

4.1 Background

Key point: LeCun worked on CNNs from 1989; the 1998 paper describes LeNet-5, used in a commercial system that read millions of cheques a day.

Yann LeCun, often called the father of CNNs, worked on convolutional networks from 1989. An early network of his read handwritten US Postal Service zip codes (LeCun et al. 1998 describe LeNet-1 as developed on this zip-code data). In 1998 he and his co-authors published **LeNet-5** (G-1080) for handwritten digit recognition. A check-reading system built on CNN character recognisers “is deployed commercially and reads several million checks per day” (LeCun et al. 1998).

4.2 The layers

Key point: conv $6@5 \times 5 \rightarrow$ avg pool $2 \times 2 \rightarrow$ conv $16@5 \times 5 \rightarrow$ avg pool $2 \times 2 \rightarrow$ flatten $\rightarrow 120 \rightarrow 84 \rightarrow 10$.

LeNet-5 expects a 32×32 greyscale image. Layer by layer (Figure 1):

1. **Convolution 1:** 6 filters of 5×5 , stride 1, no padding, tanh.
2. **Average pooling 1** (G-238): 2×2 window, stride 2. LeNet-5 uses average pooling, not max pooling.
3. **Convolution 2:** 16 filters of 5×5 , stride 1, no padding, tanh.

4. **Average pooling 2:** 2×2 window, stride 2.
5. **Flatten.**
6. **Fully connected:** 120 nodes, tanh.
7. **Fully connected:** 84 nodes, tanh.
8. **Output:** 10 nodes with softmax, one for each digit.

The activation is **tanh** (G-1947), not ReLU. ReLU is the usual choice in today's CNNs, but in 1998 tanh was the best activation function available (see [tanh](#)).

4.3 The shapes, layer by layer

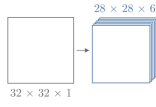
Key point: $32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5$, then $5 \times 5 \times 16 = 400$ numbers.

The sizes follow from the formulas of [strides](#): $n - f + 1$ for a convolution without padding, $\lfloor (n - 2)/2 \rfloor + 1$ for 2×2 pooling with stride 2.

Layer	Calculation	Output
Input		$32 \times 32 \times 1$
Convolution 1, 6 filters 5×5	$32 - 5 + 1 = 28$	$28 \times 28 \times 6$
Average pooling 1	$28/2 = 14$	$14 \times 14 \times 6$
Convolution 2, 16 filters 5×5	$14 - 5 + 1 = 10$	$10 \times 10 \times 16$
Average pooling 2	$10/2 = 5$	$5 \times 5 \times 16$
Flatten	$5 \times 5 \times 16$	400
Dense		120
Dense		84
Output		10

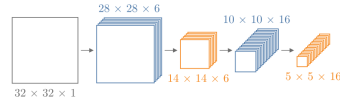
Figure 3 draws the table one layer at a time. Each block is drawn to scale, one square per map, and its calculation appears below as the block is drawn: watch the squares get smaller while the stacks get thicker.

LeNet-5: the shape after every layer



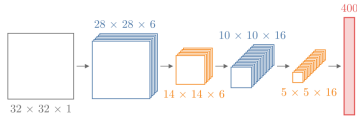
$$\text{conv } 5 \times 5, 6 \text{ filters} \\ 32 - 5 + 1 = 28$$

LeNet-5: the shape after every layer



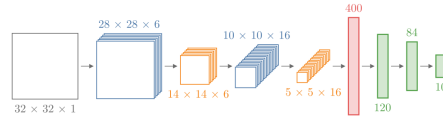
$$\text{avg pool } 2 \times 2 \\ 10 \div 2 = 5$$

LeNet-5: the shape after every layer



$$\text{flatten} \\ 5 \times 5 \times 16 = 400$$

LeNet-5: the shape after every layer



$$\text{the maps shrink } 32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5 \\ \text{while their number grows } 1 \rightarrow 6 \rightarrow 16$$

Figure 3: The shape arithmetic of LeNet-5, one layer per step. Blue: after a convolution. Orange: after average pooling. Red: the 400 flattened numbers. Green: the dense layers of 120, 84 and 10 nodes

So the first fully connected layer has 400×120 weights, the second 120×84 , and the output layer 84×10 .

The number of filters grows as we go deeper, from 6 to 16, while the filter size stays 5×5 and the height and width of the maps shrink. More filters in deeper layers is a pattern we will see again in later architectures.

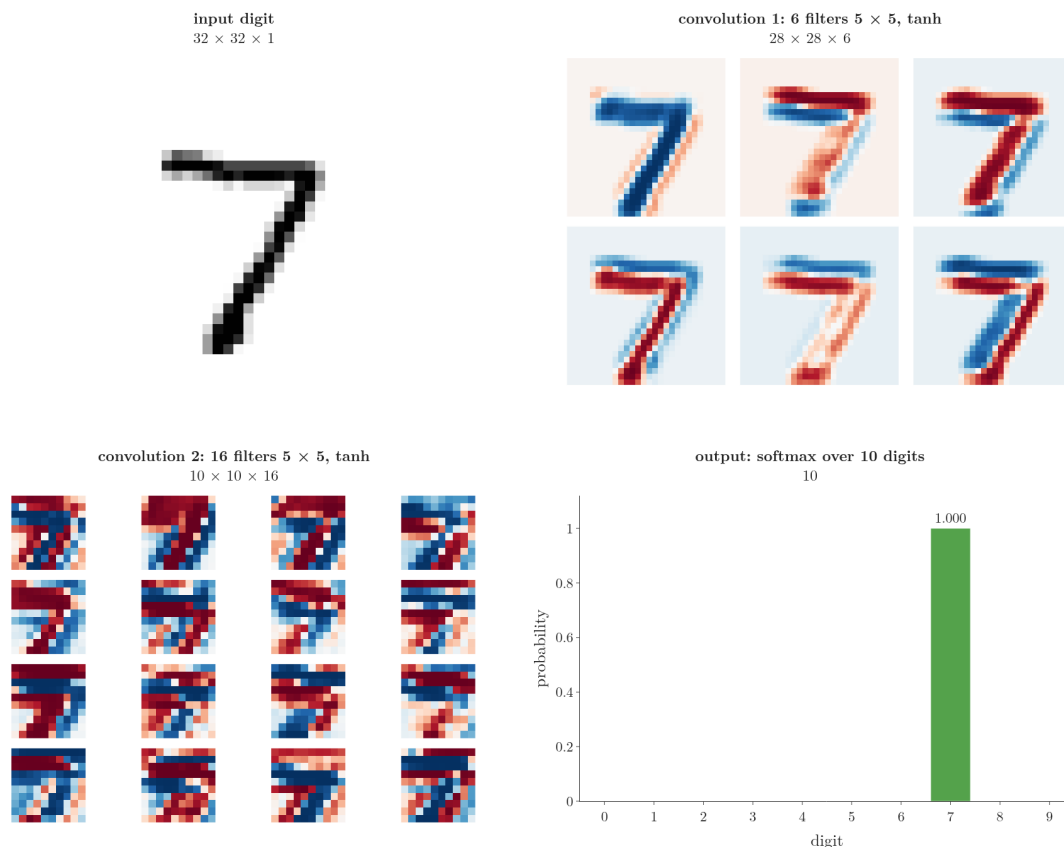


Figure 4: One MNIST test digit flowing through a trained LeNet-5 (the Keras model of section 6, seed 1, 98.4% test accuracy): the real values after every layer. Red is positive, blue negative; the dense layers are drawn as grids of cells, one per node.

In Figure 4, watch the 7 stay visible while the maps shrink from 28×28 to 5×5 and multiply from 6 to 16, then dissolve into 400 numbers once flattened; by the output, the model puts a probability of 1.000 on the digit 7.

4.4 Why “5”

Key point: The paper calls LeNet-5 a network of 7 layers, not counting the input; “5” is the version number after LeNet-1 and LeNet-4.

One common way to count reaches 5 layers: each convolution with its pooling counts as one layer, giving 2, plus the two hidden fully connected layers and the output layer. But the paper itself says “LeNet-5 comprises 7 layers, not counting the input”: C1, S2, C3, S4, C5, F6 and the output (LeCun et al. 1998). The same paper also tests LeNet-1 and LeNet-4, so the 5 is best read as a version number.

5. Counting the parameters

Key point: A convolution layer has $(\text{filter height} \times \text{width} \times \text{input channels} + 1) \times$ filters parameters. LeNet-5 in Keras has 61,706, and 48,120 of them sit in the first fully connected layer.

1. **In words:** each filter has $\text{height} \times \text{width} \times \text{input channels}$ weights plus one bias; a Dense layer has $\text{inputs} \times \text{nodes}$ weights plus one bias per node. Pooling and Flatten have none.
2. **Formula:** Here f is the filter side, c_{in} the input channels, k the number of filters, n_{in} the inputs of the Dense layer and n_{out} its nodes.

$$\text{conv} = (f \times f \times c_{\text{in}} + 1) \times k$$

$$\text{dense} = (n_{\text{in}} + 1) \times n_{\text{out}}$$

3. **Example:** convolution 1 has 5×5 filters, 1 input channel and 6 filters:

$$(5 \times 5 \times 1 + 1) \times 6 = 26 \times 6$$

$$(5 \times 5 \times 1 + 1) \times 6 = 156$$

Convolution 2 has 5×5 filters, 6 input channels and 16 filters:

$$(5 \times 5 \times 6 + 1) \times 16 = 151 \times 16$$

$$(5 \times 5 \times 6 + 1) \times 16 = 2,416$$

From `model.summary()` in the Notebook:

Layer	Output shape	Parameters
Conv2D, 6 filters 5×5	$28 \times 28 \times 6$	156
AveragePooling2D	$14 \times 14 \times 6$	0
Conv2D, 16 filters 5×5	$10 \times 10 \times 16$	2,416
AveragePooling2D	$5 \times 5 \times 16$	0
Flatten	400	0
Dense, 120	120	48,120
Dense, 84	84	10,164
Dense, 10	10	850
Total		61,706

Pooling layers have no parameters because pooling involves no training, and Flatten has none because it only reshapes. LeNet-5 has about 60,000 parameters, few for a CNN; modern CNNs have millions.

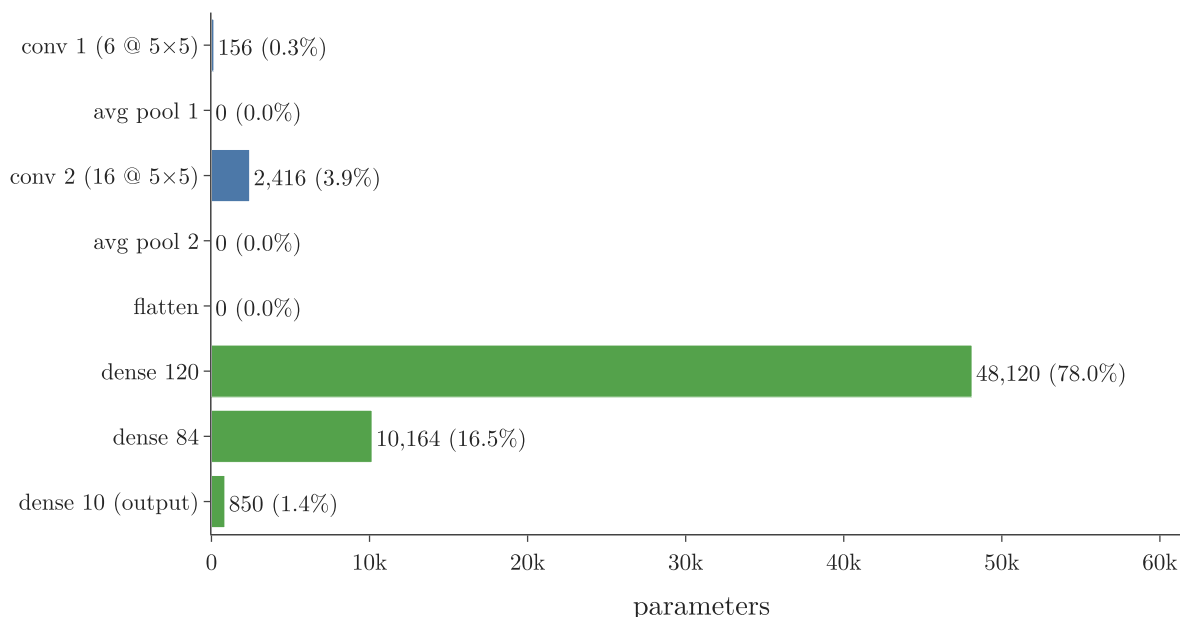


Figure 5: LeNet-5’s 61,706 parameters by layer (Notebook).

Figure 5 shows where the weights live: the two convolution layers hold 4.2% of them, and the first dense layer, which connects all 400 flattened numbers to 120 nodes, holds 78%.

Extra: The original LeNet-5 differs from the Keras version in a few details, all from LeCun et al. (1998). Its pooling layers multiply each average by a trainable coefficient and add a trainable bias (12 and 32 parameters). C3 connects each of its 16 maps to only some of S2’s 6 maps (1,516 parameters instead of 2,416). C5 is a convolution layer with 120 filters of 5×5 , which on a 5×5 input amounts to a fully connected layer. The activation is a scaled tanh, $1.7159 \tanh(Sa)$, and the output layer uses Euclidean radial basis function units instead of softmax. Adding the paper’s own counts (conv 1, pooling 1, C3, pooling 2, C5, F6) gives the “60,000 trainable free parameters” the paper states:

$$\begin{aligned}
 156 + 12 + 1,516 &= 1,684 \\
 1,684 + 32 + 48,120 &= 49,836 \\
 49,836 + 10,164 &= 60,000
 \end{aligned}$$

6. LeNet-5 in Keras

Key point: Two Conv2D + AveragePooling2D blocks, Flatten, three Dense layers. On MNIST padded to 32×32 it reaches 98.46% test accuracy after 10 epochs.

Python: LeNet-5 with today’s Keras layers.

```
model = keras.Sequential([
```

```

keras.Input(shape=(32, 32, 1)),
keras.layers.Conv2D(6, kernel_size=(5, 5), strides=1,
                    padding="valid", activation="tanh"),
keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2),
keras.layers.Conv2D(16, kernel_size=(5, 5), strides=1,
                    padding="valid", activation="tanh"),
keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2),
keras.layers.Flatten(),
keras.layers.Dense(120, activation="tanh"),
keras.layers.Dense(84, activation="tanh"),
keras.layers.Dense(10, activation="softmax"]])

```

MNIST digits are 28×28 , so the Notebook adds 2 rows and columns of zeros on every side to make them 32×32 . The paper's input was also larger than the digits, so that features at the edge of a digit can sit in the centre of a filter's view (LeCun et al. 1998). Pixels are scaled to 0–1, and the model is compiled as in [compiling and training](#) (sparse categorical cross-entropy, Adam).

Trained for 10 epochs with batch size 128, averaged over 3 seeds, LeNet-5 reaches a test accuracy of 98.46%, a test error of 1.54%. The paper reports 0.95% test error for LeNet-5 (LeCun et al. 1998), trained differently and for longer. The two-hidden-layer ANN of [the MNIST network](#) reached 97.72%.

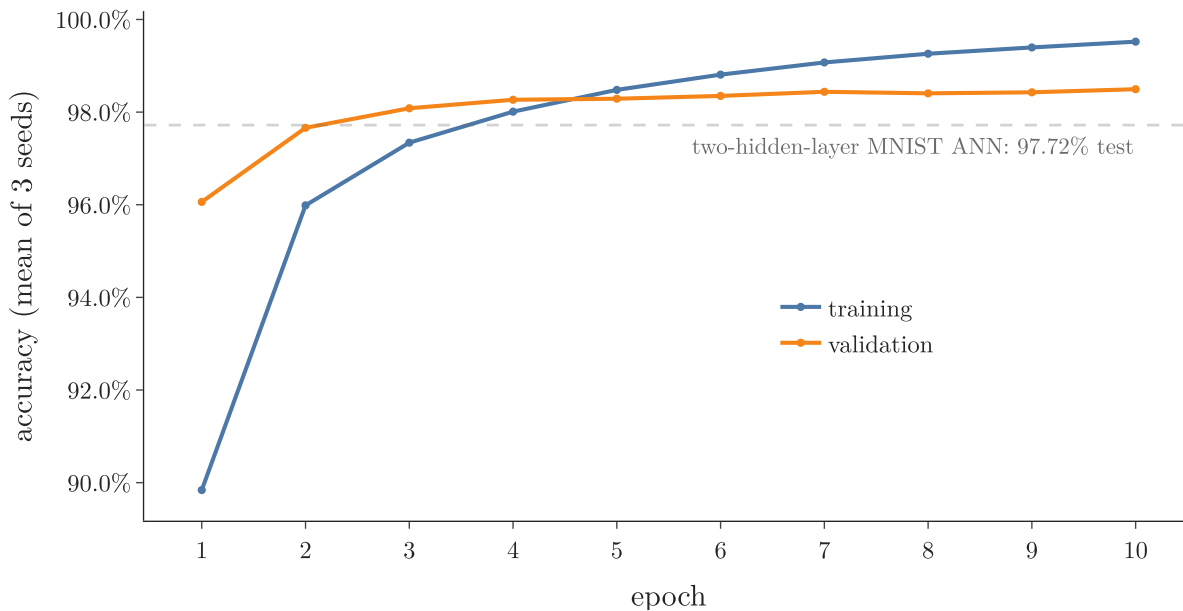


Figure 6: LeNet-5 training on MNIST: accuracy on the training images and on the validation images (the last 10% of the training set), per epoch, mean of 3 seeds. Dashed: the test accuracy of the MNIST ANN.

In Figure 6, the validation curve is just below the ANN's line at epoch 2 (97.66%), passes it at epoch 3 and levels off near 98.5%, while the training curve keeps climbing towards 99.5%.

7. Summary

	General CNN	LeNet-5
Feature extraction	$[\text{conv} + \text{ReLU} \rightarrow \text{pool}] \times M$	$2 \times [\text{conv } 5 \times 5 \text{ (6, then 16 filters)} + \text{tanh} \rightarrow \text{avg pool } 2 \times 2]$
Bridge	Flatten	$5 \times 5 \times 16 \rightarrow 400$
Classification	fully connected $\times K$	$120 \rightarrow 84 \text{ (tanh)}$
Output	sigmoid or softmax	10, softmax
Parameters	depends	61,706 in Keras (60,000 in the paper)

- A CNN = convolution and pooling blocks, Flatten, fully connected layers, output layer.
- Architectures differ in the number of layers and filters, filter size, stride, padding, activation, dropout and batch normalisation.
- LeNet-5 (LeCun et al. 1998): 32×32 input, tanh, average pooling, 6 then 16 filters of 5×5 , layers of 120, 84 and 10 nodes.
- Pooling and Flatten have no parameters; most of LeNet-5's parameters are in its first Dense layer.

8. Sources

Built from

- CampusX, “CNN Architecture | LeNet -5 Architecture”, YouTube, <https://www.youtube.com/watch?v=...>

Other references

- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. Section II.B (LeNet-5), Table I, Section III.
- Keras API documentation: `Conv2D`, `AveragePooling2D`, `Flatten` and `Dense` layers, keras.io/api/layers.
- Stanford CS231n course notes, “Convolutional Neural Networks”, section “Layer Patterns”, cs231n.github.io/convolutional-networks.

9. Key terms

Term	Meaning
CNN architecture	The sequence of layers of a CNN: convolution and pooling blocks, Flatten, fully connected layers, output
Flatten layer	A layer that turns a volume into a 1D vector; no parameters

Term	Meaning
Fully connected (Dense) layer (G-583)	A layer in which every node is connected to every input
LeNet-5	LeCun et al.'s 1998 CNN for handwritten digits: two conv-pool blocks, then 120, 84 and 10 nodes
Average pooling	Pooling that keeps the mean of each window; used in LeNet-5
ImageNet	A large image classification benchmark and competition that produced many CNN architectures